

NATIONAL DISTRICT INFRASTRUCTURE & SERVICES PORTAL

ENGINEERING IMPLEMENTATION BLUEPRINT

From Approved LLD (Volumes 1–5) to Build-Ready Delivery Plan

Pilot Districts: Nalanda / Rajgir, Bihar

Reference Standard: PM Gati Shakti (GIS-driven Planning Framework)

Source of Truth: NDISP HLD (GIS-Enriched) + LLD Volumes 1–5 + MoM v2 + Nalanda District Matrix

Status: Implementation-Ready — Derived, not redesigned, from approved documents

0. Purpose & How to Read This Blueprint

This blueprint translates the five-volume NDISP Low Level Design, the GIS-Enriched HLD, the Minutes of Meeting (v2), and the Nalanda District Matrix into a build-ready engineering plan. It does not redesign or contradict the approved LLD — it organizes the same architecture into folder structures, coding conventions, delivery sequencing, and team-facing checklists so that multiple engineering teams can build in parallel without ambiguity.

A note on the technology stack

The confirmed project stack for every team: the backend is a Python / Django / GeoDjango microservice fleet (Django REST Framework for API surfacing, View/Serializer → Service → Model/Manager layering, Django ORM migrations, Celery over RabbitMQ for async work). This blueprint applies that stack consistently across every service, folder structure, and CI step below — GeoDjango's native ORM integration with PostGIS (GeoDjango model fields, GEOSGeometry, spatial lookups) is the mechanism by which the spatial-first principles in Volumes 2–5 are realized in code.

Layer	Approved Technology (per LLD)
Frontend	React + TypeScript, Redux Toolkit, React Query, Material UI, MapLibre GL JS, OpenLayers (edit workflows), Turf.js
Backend	Django + GeoDjango microservices, Django REST Framework, DRF Serializers, REST + internal GraphQL aggregation (Strawberry-Django), Celery/RabbitMQ for async
GIS	PostgreSQL 15+ / PostGIS 3.4, GeoServer (WMS/WFS/WMTS), GDAL/OGR, PROJ, pgRouting, Cloud-Optimized GeoTIFF
Search	OpenSearch, Debezium CDC via RabbitMQ
Messaging / Cache	RabbitMQ (topic exchange `ndisp.events`), Redis (cache + sessions + Sentinel HA)
Object Storage	MinIO (erasure-coded, 4-node)
Platform	Docker (multi-stage), Kubernetes (Helm, HPA, PodDisruptionBudgets)
Observability	Prometheus, Grafana, Loki, OpenTelemetry

1. Engineering Strategy & Principles

1.1 Development Philosophy

- GIS-first, not GIS-bolted-on: PostGIS + GeoServer sit as a peer service tier to the transactional tier (LLD Vol 1, Ch. 8.1), not an add-on module. Every service that touches location data owns spatial correctness, not just the GIS Engine.
- Configuration over code for anything that varies by department or scales to 700+ districts: workflow state graphs (Ch. 15.1), asset schemas via ``mst_asset_category.field_schema`` (Ch. 13.3), and notification templates (Ch. 12.7) are all data, not deployments.
- One generic model, many departments: ``ast_facility`` + ``attributes JSONB`` (Ch. 12.3) avoids a table-per-department explosion across Health, Water, Education, Tourism, Solar, and District Assets — and the same discipline should extend to any new sector added post-pilot.
- No redesign at scale: every structural decision (Ch. 3.3 scalability boundary) is validated against the Phase 3 target (700+ districts, 100M+ assets, 500K+ concurrent citizens), not just the 2-district pilot.
- Evidence-linked outputs: every analytics or recommendation record traces to the specific spatial evidence that produced it (gap score, terrain constraint, suitability layer) — required for government audit, not optional polish.

1.2 Architecture Principles

- Microservices behind a single API Gateway; each service owns its data store slice (Ch. 8.7 Service Catalog) — no cross-service direct DB access.
- Synchronous REST for client-facing calls; internal GraphQL only for cross-service aggregation (e.g., an Admin dashboard call that stitches workflow + analytics + master-data in one round trip) — never exposed externally at pilot.
- Asynchronous-first for anything that can be: workflow transitions, ingestion completion, SLA breach, and analytics recompute all travel over the ``ndisp.events`` RabbitMQ topic exchange rather than synchronous chained calls.
- Row-Level Security as the tenancy backbone: every tenant-scoped table carries ``district_id``, enforced by PostgreSQL RLS, backstopped by an ``IsDistrictScoped`` DRF permission class at the service layer and a CQL filter injection at the GIS proxy layer — three independent enforcement points, not one.
- Contract-first APIs: OpenAPI is the source of truth; frontend hooks are generated from it (``openapi-typescript`` + React Query wrapper), and CI fails the build on a breaking contract change without a version bump.

1.3 GIS Development Principles

- SRID 4326 (WGS84) canonical for storage; reproject to 3857 only at tile-serving time — never store dual-CRS copies.
- Every raster payload is Cloud-Optimized GeoTIFF in MinIO; every raster/vector/satellite dataset is catalogued in ``gis_catalog_entry`` with provenance (``gis_data_provenance``) and version history (``gis_dataset_version_history``) — datasets are versioned, never overwritten.
- Spatial capabilities are shared, not duplicated per department: the Enterprise Spatial Analytics Engine (Ch. 17) is the single implementation of buffer analysis, overlay, network analysis, terrain/hydrology, and raster capabilities; department modules consume it, they don't reimplement it.

- GIS security is explicit: GeoServer is a shared multi-tenant instance behind `svc-gis`, which injects district-scope CQL filters server-side so a Line Department WFS-edit session cannot see or write outside its own district even though the GeoServer instance itself is shared.

1.4 Government Software Principles

- Auditability by default: every approval, tasking action, and data edit writes to an append-only, monthly-partitioned `aud\_event\_log`; a database trigger rejects UPDATE/DELETE on the table.
- Evidence over opacity: AI/analytics outputs (`anl\_recommendation`) always carry linked spatial evidence, not a bare confidence score — satisfies audit review by DM/State Admin.
- Data custodianship is explicit: `mst\_data\_custodian` (Ch. 13.5) records who owns and who to call about every department's data, sourced from the Nalanda District Matrix's custodian registry — build this into the Admin Portal, not just the database.
- Bhuvan-aligned UI, not a generic consumer app: navy/teal palette, dense information panels, left sector-navigation rail (Ch. 24.5) — deliberately chosen for official familiarity, per the Nalanda Matrix's explicit reference to Bhuvan's governance layout.
- Bilingual from day one: English + Hindi at pilot, locale-driven templates and i18n bundles so additional languages are content additions, not code changes (MoM Strategic Recommendation 1).

1.5 Scalability, Maintainability & Modularity Strategy

Dimension	Pilot (Phase 0)	National Target (Phase 3)	Structural Answer
Districts	1–2	700+	Row-level partitioning by `district_id` + RLS, not schema-per-tenant (Ch. 8.6)
Assets	~50,000	100,000,000+	GiST/GIN indexing, declarative table partitioning by `state_id` on high-volume tables
Concurrent citizens	~500	500,000+	Stateless services on HPA; CDN-fronted static + WMTS tiles
Departments	5–6	20+ (state-specific)	`mst_department` + `attributes JSONB` — new sector = master-data row, not redeploy
Map tile requests/sec	~50	50,000+	Pre-generated MVT for static layers, Redis+CDN cache for dynamic layers (Ch. 28.6)

1.6 Security-First Approach (summary — full detail in Part 10)

- Target OWASP ASVS Level 2 as the floor, not the ceiling.
- Defense in depth on tenancy: RLS (DB) + IsDistrictScoped permission class (service) + Gateway RBAC (edge) — any single layer failing does not leak cross-district data.
- Encryption at rest (AES-256) and in transit (TLS 1.2+) everywhere; secrets never in source control or images.

2. Repository Strategy

2.1 Why a Monorepo

The approved LLD already assumes shared, generated contracts across surfaces — typed API hooks generated from OpenAPI (Ch. 24.4), a shared ``ui-components`/`map-core`/`i18n`` package set (Ch. 24.1), and a single ``ndisp.events`` event vocabulary consumed by many services (Ch. 18.2, Ch. 25.6). A polyrepo would force these into published, versioned packages far earlier than the pilot needs, slowing every cross-cutting change (a new locale, a new workflow type, a new shared component). A monorepo keeps contract changes atomic: one PR can update an OpenAPI spec, its generated client hooks, and the consuming component in the same review.

- Atomic cross-cutting changes: a shared type or event schema change lands in one commit, not a coordinated multi-repo release.
- Single CI pipeline enforces the OpenAPI contract-diff check (Ch. 26.3) across every consumer at once.
- Consistent tooling across a polyglot monorepo: Poetry workspaces + ruff/black/mypy for the Django services, TypeScript config + ESLint for the frontend apps, both driven from a single Turborepo/Nx task graph at the root — instead of per-repo drift across 20+ services.
- 700+ district scale is a data-partitioning problem (Ch. 8.6), not a code-partitioning problem — services don't multiply per district, so repo count doesn't need to either.

2.2 Top-Level Folder Hierarchy

```
ndisp/
├── apps/                                # deployable frontend applications
│   ├── citizen-portal/
│   ├── admin-portal/
│   ├── linedept-portal/
│   └── mobile-app/                     # React Native — Field Engineer + Citizen views
├── services/                           # deployable backend microservices (Django/GeoDjango)
│   ├── svc-auth/
│   ├── svc-gis/
│   ├── svc-workflow/
│   ├── svc-analytics/
│   ├── svc-notification/
│   ├── svc-master-data/
│   ├── svc-asset/
│   ├── svc-reporting/
│   ├── svc-search/
│   ├── svc-document/
│   ├── svc-audit/
│   └── api-gateway/
├── packages/                           # shared libraries, published internally via workspace protocol
│   ├── ui-components/                 # MUI + Bhuvan-aligned design tokens
│   ├── map-core/                      # MapLibre / OpenLayers wrappers, layer manager
│   ├── api-client/                   # generated OpenAPI hooks (React Query)
│   ├── i18n/                         # EN/HI locale bundles
│   ├── event-contracts/              # ndisp.events schema + typed publishers/consumers
│   ├── geo-validators/               # shared GeoJSON/SRID/ring-closure validation (backend + CI)
│   └── utils/
├── gis/                                # GIS platform configuration, not application code
│   ├── geoserver/                   # workspace/layer/style (SLD) definitions as config
│   ├── styles/                      # SLD / MapLibre style specs
│   └── datasets/                    # DEM/LULC/NDVI processing job definitions (Ch. 12.11–12.13)
├── database/
│   ├── migrations/                  # per-service Django migrations (makemigrations/migrate)
│   ├── seed/                       # mst_* reference data seed scripts
│   └── erd/                        # exported ERDs (Ch. 12.15 as source diagram)
```

— deployment/	
— helm/	# one chart per service + umbrella chart
— k8s/	# namespace, network policy, PDB manifests
— environments/	# ndisp-dev / ndisp-staging / ndisp-prod values
— monitoring/	
— grafana/	# dashboards incl. "GIS Health" composite (Ch. 22.1)
— prometheus/	# scrape configs, alert rules
— loki/	
— tools/	# codegen, contract-diff checker, dev-environment scripts
— scripts/	# one-off ops scripts (backfills, reindex jobs)
— tests/	
— integration/	# docker-compose based cross-service tests (Ch. 29.2)
— performance/	# k6/Gatling scenarios (Ch. 29.4)
— docs/	# this blueprint + ADRs
— examples/	# reference implementations for new-service scaffolding

2.3 Shared Packages — Ownership & Contract

Package	Owns	Consumed By
ui-components	MUI theme, Bhuvan-aligned tokens, no business logic	All apps/
map-core	MapLibre/OpenLayers wrappers, Layer Manager, vector-tile client	All apps/, mobile-app
api-client	Generated typed hooks from OpenAPI (Ch. 24.4)	All apps/
event-contracts	`ndisp.events` routing-key + payload schemas (Ch. 18.2, 25.6)	All services/
geo-validators	GeoJSON/SRID/ring-closure validation rules, defined once and mirrored as a TS package (apps/) and a Python validator module (services/) — same rule set, kept in sync via a shared JSON Schema (Ch. 27.6)	apps/, services/
i18n	EN/HI locale bundles, ICU tokenizer config for search parity (Ch. 19.3)	All apps/

2.4 Versioning & Dependency Strategy

- Internal packages consumed via workspace protocol (no publish step) at pilot; promote to a private registry only if a package needs to be consumed outside this monorepo (e.g., a future state-level integration).
- Service-to-service contracts versioned via URL path (`/api/v1/...`, Ch. 23.1); a breaking change requires a `v2` path, never an in-place breaking edit — enforced by the CI contract-diff check.
- Database migrations are per-service and forward-only in production; rollback is by forward-fix migration, matching the append-only audit posture (Ch. 12.8).
- Dependency upgrades run through a single Renovate/Dependabot configuration at the repo root so 20+ services don't drift on Python/Django/GeoDjango minor versions on the backend, or Node/React minor versions on the frontend.

3. Frontend Architecture

3.1 Per-App Folder Structure

```
apps/<portal>/src/  
  modules/  
    health/      { components/, hooks/, pages/, api.ts }  
    water/       { components/, hooks/, pages/, api.ts }  
    education/   { components/, hooks/, pages/, api.ts }  
    tourism/     { components/, hooks/, pages/, api.ts }  
    solar/       { components/, hooks/, pages/, api.ts }  
    assets/      { components/, hooks/, pages/, api.ts }  
  components/   # app-shell: header, nav, layout  
  pages/        # route-level containers  
  hooks/        # cross-module hooks  
  context/      # AuthContext, MapContext, LocaleContext  
  store/        # Redux Toolkit slices  
  theme/        # MUI theme, Bhuvan-aligned tokens  
  routes.tsx
```

Folder structure is normative per LLD Vol 5, Ch. 24.1 — reproduced here as the binding contract for all frontend teams.

3.2 State Management Split

State Class	Owner	Rule
Auth, active district/role scope, notifications	Redux Toolkit	Cross-cutting app state only — not server data
Facility lists, workflow instances, any server-fetched data	React Query	Cache-first; invalidate on mutation success and relevant WebSocket/notification events
Map viewport (pan/zoom/center)	Local MapContext	Never in Redux — avoids re-render storms on pan/zoom (Ch. 24.2)

3.3 Component Layering

Atomic-ish, one direction only:

- ui-components — buttons, cards, form fields. No business logic, no API calls.
- module components — e.g. FacilityCard, GapScoreLegend. Composed from ui-components, aware of domain shape.
- pages — compose components + hooks + routing. This is the only layer allowed to orchestrate multiple hooks/queries.

3.4 Data Fetching & Hooks

Typed hooks generated from the OpenAPI spec via openapi-typescript, wrapped in a thin React Query layer — e.g. useAssets(filters), useCreateGrievance(). This keeps frontend and backend in sync automatically: CI's contract-diff check fails the build if a backend change breaks a generated hook signature without a version bump (Ch. 24.4, 26.3).

3.5 Theming — Bhuvan-Aligned Design Tokens

Per the Nalanda Matrix's explicit reference to Bhuvan's governance layout (bhuvan.nrsc.gov.in/ngmaps/governance), the shared theme package defines a navy/teal primary palette, dense

information panels, and a left sector-navigation rail with quick-stat chips — deliberately consistent with government-official UI familiarity rather than a generic consumer-app aesthetic (Ch. 24.5). Treat this as a hard constraint on any visual design work, not a starting suggestion.

3.6 Routing, Role Guards & Lazy Loading

- Route-based code splitting per module via `React.lazy` (Ch. 24.6).
- Role-aware route guards read `AuthContext.permissions` to hide/show Admin vs. Line Department vs. Citizen routes — required even within the mobile app's shared shell, since it serves both Field Engineer and Citizen views.
- Map layers load on-demand as toggled by the user, not all at initial load (Ch. 28.3).

3.7 Module Breakdown — Standard Contents

Every department module (Health, Water, Education, Tourism, Solar, District Assets) and every cross-cutting module (Auth, Dashboard, GIS, Reports, Settings) ships the same five artefacts:

- `components/` — module-specific presentational + container components
- `hooks/` — generated + composed data hooks for that module's endpoints
- `pages/` — route-level screens registered in `routes.tsx`
- `api.ts` — thin typed wrapper re-exporting the relevant api-client hooks for this module
- Route-level permission declarations consumed by the role guard (Ch. 3.6)

4. Backend Architecture (Django / GeoDjango Microservices)

4.1 Per-Service Folder Structure

```
services/<svc-name>/
manage.py
config/                                # Django project package for this service
  settings/
    base.py
    dev.py / staging.py / prod.py
  urls.py                             # root URLconf, includes each app's urls.py
  asgi.py
  celery.py                           # Celery app bound to this service, RabbitMQ broker
common/
  middleware/                         # AuditLogMiddleware, RequestIdMiddleware, envelope middleware
  exceptions/                         # custom APIException classes -> {data, meta, errors} envelope
  permissions/                        # IsAuthenticatedJWT, HasRolePermission, IsDistrictScoped
  pagination/                         # standard PageNumberPagination subclass
apps/
  <domain>/
    models.py                         # GeoDjango models (spatial fields where relevant)
    views.py                          # DRF ViewSets/APIViews - HTTP concerns only
    services.py                       # business logic, transactions (service layer)
    selectors.py                      # data access, PostGIS-aware queryset builders
    serializers/
      create_<domain>_serializer.py
      update_<domain>_serializer.py
    permissions.py                    # domain-specific DRF permission classes
    tasks.py                          # Celery tasks for async/background work
    events/
      <domain>_<event>.py              # domain event publisher -> ndisp.events (RabbitMQ)
    urls.py                           # app-level DRF router registration
    migrations/                       # Django migrations (per app)
  database/                           # seed data management commands, cross-app fixtures
```

4.2 Layering Convention (strict, non-negotiable)

View/ViewSet (HTTP concerns only) → Service (business logic, transactions) → Selector (data access, PostGIS-aware queryset builders). This is the standard Django "thin views, fat services" discipline applied strictly: DRF ViewSets never call the ORM directly, and spatial-query complexity must stay out of views and be unit-testable in isolation at the selector layer (Ch. 25.2).

4.3 Serializers & Validation

DRF Serializers on every input/output boundary — GeoDjango's GeoFeatureModelSerializer (django-rest-framework-gis) is the default for spatial payloads. A shared GeoPointField / GeoPolygonField validator (from packages/geo-validators, mirrored as a Python validator module reused across services) checks GeoJSON structure and SRID before a payload ever reaches the selector layer — malformed geometry must never reach PostGIS (Ch. 25.3, Ch. 27.6).

4.4 Authentication & Permission Classes (applied in this order)

Class	Responsibility
JWTAuthentication	Verifies token signature and expiry
HasRolePermission	Checks a @require_permission('resource:action') decorator / permission_classes entry against the JWT permissions[] claim

Class	Responsibility
IsDistrictScoped	Filters every queryset by <code>.filter(district_id=request.user.district_scope)</code> at the view/selector boundary — backstops PostgreSQL RLS (defense in depth)

4.5 Middleware & Exception Handling

- AuditLogMiddleware + a post_save/post_delete Django signal receiver — after every successful mutating request, publishes {entity_type, entity_id, before, after} to RabbitMQ asynchronously via a Celery task; never blocks the user-facing transaction.
- A custom DRF exception_handler (registered as EXCEPTION_HANDLER in settings) — maps every domain exception to the standard {data, meta, errors} envelope with correct HTTP status and a machine-readable error_code for client-side i18n.

4.6 Events & Queues

RabbitMQ topic exchange ndisp.events, routing key pattern {domain}.{event} (e.g. workflow.transitioned, asset.ingested, grievance.sla_breached), published via Celery tasks (using kombu/celery's AMQP integration) triggered from Django signal receivers on the relevant model. Each service declares its own queue bound to relevant routing keys via a Celery worker; dead-letter queues are configured per consumer with alerting on DLQ depth > 0 (Ch. 18.2, 25.6).

4.7 Engine Responsibilities — Service Catalog

Service	Responsibility	Primary Data Store
svc-auth	Authentication, JWT/refresh tokens, OIDC/SSO	PostgreSQL (users, roles) + Redis (sessions)
svc-gis	Spatial queries, tile-serving orchestration, geocoding, GeoServer proxy	PostGIS, GeoServer
svc-workflow	Generic state-machine engine: proposal → approval → tasking → completion	PostgreSQL
svc-analytics	Enterprise Spatial Analytics Engine: deficit detection, gap scores, SDSS recommendations, canonical spatial query library (Ch. 6.9), hazard-safety recomputation (Ch. 6.10)	PostGIS (read), Redis (cache)
svc-notification	SMS/Email/WhatsApp/Push/Portal dispatch, channel-agnostic	PostgreSQL, RabbitMQ
svc-master-data	State/District/Block/Village hierarchy, department/scheme reference data	PostgreSQL
svc-asset	Department asset CRUD, geo-tagging, generic Asset model	PostGIS
svc-reporting	PDF/Excel/CSV/GIS report generation	MinIO (output storage)
svc-search	Global + spatial search, autocomplete, geospatial catalog search	OpenSearch
svc-document	Uploads, geo-tagged images, OCR, versioning, digital signature refs	MinIO, PostgreSQL (metadata)
svc-audit	Immutable, append-only audit logging	PostgreSQL (partitioned, write-once trigger)

4.8 Toolchain Bindings Inside Existing Services (no new microservices)

GDAL and OGR are bound natively via GeoDjango's own GDAL Python bindings (django.contrib.gis.gdal) and rasterio for raster-heavy jobs; PROJ is used through pyproj for any transform not already covered by GeoDjango's SRID-aware fields; pgRouting is invoked as SQL functions through the Django ORM's raw/RunSQL

escape hatch inside svc-gis / svc-analytics selectors. These run in-process within the existing Django services (with a GDAL-enabled base image), or as a dedicated Celery worker pool for long-running raster jobs — toolchain additions inside existing service boundaries, not new deployable units (Ch. 25.7).

5. Database Design Strategy

5.1 Design Principles (binding for every migration)

- PostgreSQL 15+ with PostGIS 3.4. SRID 4326 canonical for all geometry columns; reproject to 3857 only at tile-serving time.
- Every table carries: id UUID PK (gen_random_uuid()), created_at, updated_at, created_by, updated_by, and soft-delete via is_deleted BOOLEAN — hard delete reserved only for GDPR-class erasure requests.
- Every tenant-scoped table carries district_id UUID NOT NULL REFERENCES mst_district(id), enforced via PostgreSQL Row-Level Security.
- Naming convention is mandatory and prefix-based: mst_* (master), txn_* (transactional/workflow), ast_* (assets/GIS), aud_* (audit), ntf_* (notification), anl_* (analytics), doc_* (documents), rbac_* (permissions), gis_* (raster/terrain/catalog).

5.2 Schema Groups

Prefix	Purpose	Representative Tables
mst_*	Master / reference data	mst_state, mst_district, mst_subdivision, mst_block, mst_village_ward, mst_department, mst_scheme, mst_asset_category, mst_user, mst_data_custodian
ast_*	Assets / GIS features	ast_facility, ast_ingestion_batch, ast_facility_history (SCD Type-2), ast_road_network, ast_pipeline_network
txn_*	Transactional / workflow	txn_workflow_instance, txn_workflow_transition, txn_citizen_grievance, txn_proposal, txn_budget_approval
anl_*	Analytics outputs	anl_gap_score, anl_recommendation
doc_*	Documents / media	doc_file
ntf_*	Notifications	ntf_template, ntf_dispatch_log
aud_*	Audit (append-only)	aud_event_log (monthly partitioned)
rbac_*	Access control	rbac_role, rbac_permission, rbac_role_permission, rbac_user_district_scope
gis_*	Raster / terrain / catalog	gis_dem_dataset, gis_lulc_dataset, gis_ndvi_dataset, gis_watershed, gis_drainage_network, gis_soil_dataset, gis_rainfall_dataset, gis_groundwater_dataset, gis_satellite_imagery, gis_catalog_entry, gis_processing_job, gis_dataset_version_history, gis_data_provenance, gis_spatial_quality_metric

5.3 Indexing Strategy

- GiST indexes on every geometry column; the primary structural index for all spatial predicates (ST_DWithin, ST_Distance, ST_Contains).
- GIN indexes on JSONB attributes columns (ast_facility.attributes) so department-specific fields remain query-indexable without a table-per-department design.
- BRIN indexes on time-partitioned audit/asset-history tables — cheap, effective on monotonically-increasing timestamp columns at high write volume.
- btree(district_id, department_id) composite indexes on high-traffic filtered tables such as ast_facility.
- REINDEX CONCURRENTLY scheduled during low-traffic windows for the largest tables (ast_facility, anl_gap_score) at national scale; autovacuum tuned specifically for high-write geometry tables.

5.4 Partitioning

- Declarative table partitioning by `state_id` applied to high-volume tables (assets, audit log) for query performance at 700+ district scale.
- `aud_event_log` is monthly range-partitioned on `occurred_at`.
- Row-level tenancy (`district_id` + RLS) is the primary multi-tenancy mechanism — schema-per-tenant was explicitly rejected because 700+ schemas would break connection pooling and migration tooling at national scale (Ch. 8.6).

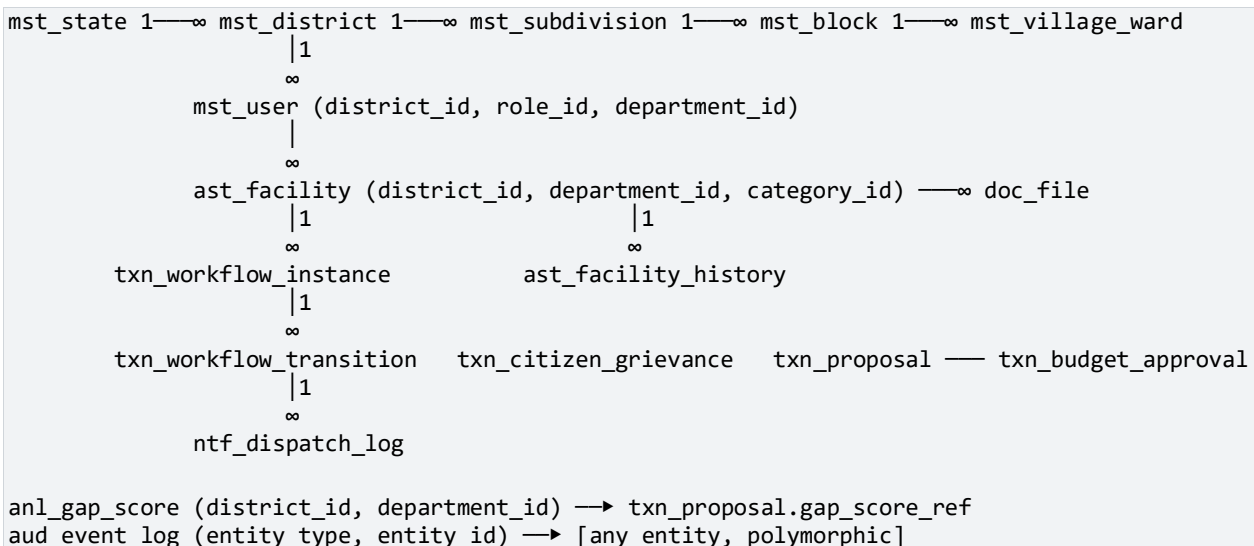
5.5 Audit & Versioning Strategy

- Soft delete via `is_deleted` on almost every table; append-only `aud_event_log` captures create/update/delete/approve/reject/login with `before_state/after_state` JSONB snapshots.
- `ast_facility_history` implements SCD Type-2 (`valid_from/valid_to/snapshot`) for asset rollback and lifecycle tracking, not just point-in-time audit.
- Geospatial datasets are versioned via `gis_dataset_version_history` — raster/vector datasets are never overwritten, supporting historical/change-detection analysis.

5.6 Migration Strategy

- Per-service Django migrations (`makemigrations/migrate`), forward-only in production; rollback is by forward-fix migration, matching the append-only audit posture.
- `mst_asset_category.field_schema` (JSONB) and workflow-type state graphs are configuration rows, not migrations — adding a new asset type or workflow type must never require a schema migration or service redeploy (Ch. 13.3, 15.1). Treat any PR that adds a new department by writing a migration as a design smell.

5.7 Entity Relationship Overview



6. GIS Architecture

6.1 Layer Model

Layer Type	Examples	Storage
Administrative	State, District, Subdivision, Block, Village/Ward boundaries	PostGIS MultiPolygon, SRID 4326
Raster	Satellite base imagery (Bhuvan/NRSC), elevation	GeoServer raster store, GeoTIFF/COG
Vector	Facilities (points), roads/pipelines (lines), catchments (polygons)	PostGIS, served as vector tiles
Derived Analytics	Deficit heatmaps, service-radius buffers, hotspots	PostGIS materialized views, regenerated by Analytics Engine

6.2 Shapefile / CSV / Raster Ingestion (Ch. 11)

A single staged pipeline handles every incoming spatial dataset, department CSV or raster alike:

```
CSV/Excel Upload → Schema Validation → Geocoding → Deduplication →  
Transformation (canonical model) → Staging Table → QA Review (DEO) →  
Publish to svc-asset (PostGIS) → Audit Log → Analytics Recompute Trigger
```

- Schema Validation: column presence, datatype, mandatory fields, enum domains — failures quarantined with row-level reason codes returned to the uploader.
- Geocoding: lat/long validated within district boundary via ST_Contains; address-only rows geocoded with a confidence score attached.
- Deduplication: fuzzy match on (name + geometry proximity $\leq 50\text{m}$) against existing ast_* records.
- Staging: written to stg_asset_upload for DEO review — nothing goes live without explicit QA confirmation at pilot.
- Raster ingestion (DEM/LULC/NDVI/satellite imagery) follows the same staging → QA → publish discipline via gis_processing_job, converging on Cloud-Optimized GeoTIFF in MinIO and a gis_catalog_entry record (Ch. 12.11–12.13).

6.3 GeoServer — Layer Publishing, Styling, Caching

- GeoServer exposes WMS (map images), WFS (feature data for admin editing tools), and WMTS (cached base tiles) — all OGC-compliant for NSDI/PM GatiShakti interoperability without adapters.
- svc-gis proxies GeoServer end-to-end, injecting district-scope CQL filters server-side based on the caller's JWT district_id — GeoServer itself remains a single shared multi-tenant instance.
- MapLibre GL JS consumes vector tiles directly for citizen/admin maps; OpenLayers is used specifically where WMS/WFS editing widgets are required (Line Department asset editing UI).
- Vector tiles: pre-generated via ST_AsMVT for static/slow-changing layers (administrative boundaries); on-the-fly MVT generation with Redis caching for dynamic layers (facility points, gap scores) — 5-minute TTL for dynamic, 24h for static.

6.4 Spatial Query Strategy

- Radius search / 'nearest N': ST_DWithin / ST_Distance with a bounding-box pre-filter for performance at 100M+ point scale.

- Isochrone / travel-time: pgRouting network analysis against `ast_road_network`, chosen over an external routing engine (e.g. OSRM) to keep the spatial stack within the existing PostGIS deployment — OSRM remains a swappable fallback if road-network coverage proves insufficient.
- Hotspot detection: kernel density estimation via PostGIS `ST_ClusterKMeans` / custom SQL over citizen-grievance points and disease-outbreak reports.

6.5 Spatial Analysis Capability Catalog (Enterprise Spatial Analytics Engine, Ch. 17.6)

Category	Capability	Primary Consumer(s)
Proximity	Buffer analysis	Water (recharge structures), Tourism (heritage buffers)
Proximity	Service-radius / nearest-facility	Health, Education
Proximity	Isochrone / travel-time (pgRouting)	Health (emergency reachability), Education (catchment)
Overlay	Overlay analysis	All departments (gap-score composition)
Overlay	Weighted overlay / MCDA suitability	Solar (site selection), Water (recharge suitability)
Network	Network / route analysis	Tourism (corridor planning), Health (ambulance routing)
Density	Heatmaps / kernel density	Tourism (visitor density), Health (outbreak hotspots)
Density	Hotspot detection / clustering	Health, Water (grievance clustering)
Terrain	DEM processing, slope/aspect/contour	Water (hydrology), Solar (shadow/orientation)
Terrain	Watershed delineation, drainage-network	Water
Raster	NDVI computation	Water, District Assets
Raster	LULC classification	Solar (site exclusion), Water (runoff modeling)
Raster	Satellite change detection	District Assets, Water (water-body extent)
Locational Optimization	Location-allocation	Health, Education (facility siting)
Locational Optimization	Spatial interpolation	Water (rainfall/groundwater surfaces)
Governance	Geofencing	Tourism (heritage buffers), District Assets

6.6 Spatial Decision Support System (SDSS)

Pilot ships a rules-based MCDA engine (Ch. 17.5): weighted overlay of `anl_gap_score`, terrain constraints, and department budget-approval history, producing `anl_recommendation` records that always carry linked spatial evidence — never a bare confidence score. The recommendation API contract (Ch. 23.4a) is deliberately stable between this rules-based implementation and a future ML-ranked version reserved for state rollout, so upgrading the model later requires no API-consumer changes.

Decision Class	Spatial Evidence Combined
Recommend PHC/facility location	Population-coverage gap, travel-time isochrone, existing service radius
Recommend water-harvesting structure site	Watershed delineation, drainage accumulation, groundwater recharge potential, soil/slope suitability
Recommend solar site (roof or farm)	Irradiance surface, roof suitability/shadow analysis, LULC exclusion, transmission proximity
Recommend tourism circuit	Visitor density, network analysis, heritage-buffer constraints
Recommend school location	Catchment analysis, population served, travel distance
Recommend road/connectivity priority	Network analysis, settlement accessibility gap

Decision Class	Spatial Evidence Combined
Recommend infrastructure investment priority	Weighted multi-criteria overlay across all department gap scores

6.7 Remote Sensing Integration

Source	Role
NRSC / Bhuvan	Primary national satellite-imagery and thematic-layer source (LULC, administrative reference)
Sentinel (ESA Copernicus)	Medium-resolution multispectral imagery for NDVI, seasonal change detection
Landsat (USGS)	Long time-series imagery for historical LULC/water-body change detection

Processing chain: raw satellite imagery → gis_processing_job (NDVI compute / LULC classify / change detect) → derived product → catalogued in gis_catalog_entry → surfaced via GeoServer as a standard map layer and as an input to department suitability models.

6.8 Master Layer Catalogue (Client-Confirmed — Queries_for_Nalanda.xlsx)

The client-supplied Queries_for_Nalanda.xlsx (Layer sheet) enumerates 49 named layers with confirmed geometry types. This is the ratified seed list for gis_catalog_entry / svc-asset layer registration at pilot — every layer below is registered as a first-class layer in the Layer Model (§6.1) at the category shown, superseding any placeholder layer naming used earlier in this document.

Category	Layers (Geometry Type)
Administrative & Reference	District_boundary (polygon), Block_boundary (polygon), Headquarters (point)
Health	Hospital (point), Community_Health_centre (point), Primary_Health_centre (point), Dispensary (point), Blood_Bank (point), Veterinary_Hospital (point)
Water Resources	Well (point), Tubewell (point), Spring (point), Waterbody (polygon), River (polygon), River_line (line), Canal_poly (polygon), GroundWater_Potential (polygon), Water_Table_Contour (line)
Education	School (point), Collage (point) [client spelling — see note], University (point)
Tourism & Heritage	Places_of_Tourist_Interest (point), Temple (point), Mosque (point), Church (point), Circuit_house (point), Inspection_Bungalow (point)
Transport & Connectivity	National_Highway (line), State_Highway (line), Other_Roads (line), Railway_line (line), Railway_station (point)
Public Amenities & Admin	PoliceStation (point), PostOffice (point), Market (point), Industry (point), Bank (point)
Population & Socioeconomic	Rural_population (polygon), Urban_population (polygon), Occupational_Structure (point)
Hazard (new category)	Flood_hazard (polygon), Earthquake (polygon), Wind_Hazard (polygon)
Terrain & Climate	Relief (polygon), Slope (polygon), Rocks (polygon), Soil (polygon), Isohyet_Lines (line), Isotherm_Lines (line), Rainfall_Zone (polygon)
Land Use & Forest	Landuse_NALANDA_NRSC (polygon), RF_PF_boundary (line)

Note (assumption A-GIS-3): the client's Layer sheet spells the higher-education layer “Collage.” The canonical Django model/table name normalizes this to college; the CSV/shapefile ingestion pipeline (§6.2) accepts the client's original header via a column-alias map so no re-upload is required. Flagged for client confirmation, not silently decided.

6.9 Canonical Spatial Query Library (Client-Confirmed — Queries_for_Nalanda.xlsx)

Queries_for_Nalanda.xlsx (Sheet5) specifies 34 fully defined spatial queries across three perspectives — Citizens, Government Administration, Line Departments — and five sectors — Health, Water, Tourism, Education, Solar

— plus 6 placeholder rows the client reserved for additional queries at UAT (Health, Water, Solar). Every query maps onto an existing capability already designed in the Spatial Analysis Capability Catalog (§6.5); none requires a new spatial-analysis primitive, only parameterization and cataloguing of combinations already in scope.

Each row below is registered as a QueryDefinition record (query_id, perspective, sector, layers_required[], query_pattern, portal) consumed by svc-analytics, and exposed uniformly at GET /api/v1/queries (list/discover) and POST /api/v1/queries/{query_id}/execute (run with user-supplied parameters such as location, radius, threshold) — see §7.2. Portals call these registered definitions rather than re-deriving spatial logic per screen, so a query fixed once is fixed everywhere it is surfaced.

Perspective	Sector	Query	Primary Pattern (§6.5)
Citizens	Health	Nearest health facility finder	Nearest-facility + network route
Citizens	Health	Nearby hospital and blood bank search	Buffer (10 km) + rank by distance
Citizens	Health	Safe health facility during disaster	Hazard-safety filter (§6.10)
Citizens	Water	Nearby drinking water source locator	Buffer (2 km)
Citizens	Water	Groundwater potential around my area	Overlay (point-in-polygon) + nearest source
Citizens	Tourism	Nearby tourist and religious places	Buffer (10/15 km)
Citizens	Tourism	Tourist place with accommodation access	Buffer (5/10 km) + road-proximity filter
Citizens	Tourism	Safe tourist destination finder	Hazard-safety filter + road-access filter
Citizens	Education	Accessible schools near my home	Buffer (3 km) + attribute filter
Citizens	Education	Safe and connected education facilities	Hazard-safety filter + road-proximity
Citizens	Education	Student convenience around institutions	Nearest-facility + co-located services
Citizens	Solar	Solar-ready public institutions nearby	Road-access filter + hazard-safety filter
Govt. Administration	Health	Block-wise health service gap	Overlay — population-vs-coverage gap
Govt. Administration	Health	Hazard exposure of health facilities	Hazard-safety filter (intersect)
Govt. Administration	Water	Population versus water source access gap	Overlay — population-vs-coverage gap
Govt. Administration	Water	Flood-vulnerable drinking water points	Hazard-safety filter (intersect/near)
Govt. Administration	Tourism	Tourism amenities gap analysis	Overlay — amenity gap
Govt. Administration	Tourism	Tourism hazard risk mapping	Hazard-safety filter (intersect)
Govt. Administration	Tourism	Transport access priority for tourism	Network — distance-threshold ranking
Govt. Administration	Education	Education hazard and connectivity review	Hazard-safety filter + road-distance
Govt. Administration	Education	Block-wise education expansion priority	Overlay — gap analysis + priority score
Govt. Administration	Solar	Solar readiness of public institutions	Hazard-safety filter + institution filter
Govt. Administration	Solar	Ground-mounted solar suitability screening	Weighted overlay / MCDA suitability

Perspective	Sector	Query	Primary Pattern (§6.5)
Line Departments	Health	New health facility planning support	Overlay — gap analysis (siting)
Line Departments	Health	Disaster-ready health network planning	Hazard-safety filter + network redundancy
Line Departments	Health	Blood bank/hospital logistics optimization	Network — travel-time/distance threshold
Line Departments	Water	New tubewell/well support	Weighted overlay suitability + gap analysis
Line Departments	Water	Groundwater stress and dependency zones	Overlay — stress/dependency index
Line Departments	Tourism	Heritage circuit design support	Density/clustering + network continuity
Line Departments	Tourism	Tourism safety and advisory planning	Hazard-safety filter + terrain risk
Line Departments	Education	School expansion and upgrade planning	Overlay — gap analysis + demand threshold
Line Departments	Education	School resilience planning	Hazard-safety filter + access filter
Line Departments	Solar	Institutions for rooftop solar install	Priority scoring + hazard-safety filter
Line Departments	Solar	Cross-sector solar convergence planning	Multi-sector weighted suitability score

6.10 Hazard-Safety Attribute Pattern (New — required by Queries_for_Nalanda.xlsx)

The client's query set introduces hazard-safety as a cross-cutting, computed attribute on every point-facility layer used in citizen and administrative decisions (Hospital, Community_Health_centre, Primary_Health_centre, Dispensary, School, Collage, University, Places_of_Tourist_Interest, Temple, Mosque, Church, and public institutions screened for solar) — not previously modeled in this blueprint. Pattern, added uniformly across the affected GeoDjango models:

- Each eligible facility model gains a computed field `hazard_safe` (nullable BooleanField until first computed) and `hazard_flags` (JSONField, e.g. {"flood": false, "earthquake": true, "wind": false}).
- A Celery task `recompute_hazard_safety(facility_type, facility_id)` runs `ST_Intersects(facility.geom, hazard.geom)` against `Flood_hazard`, `Earthquake`, and `Wind_Hazard`, triggered on (a) facility create/update via a Django `post_save` signal, and (b) hazard-layer republish through the ingestion pipeline (§6.2).
- A nightly Celery Beat job `recompute_hazard_safety_all()` reconciles drift for hazard layers updated out-of-band (e.g. a revised flood-hazard shapefile from IMD/CWC), so `hazard_safe` never silently goes stale.
- `hazard_safe` / `hazard_flags` are exposed on every affected facility serializer so citizen, admin, and line-department queries (§6.9) filter on a precomputed field — no per-request spatial join — the same diagnostic-precompute discipline already used for `anl_gap_score` (§6.6).

7. API Architecture

7.1 Conventions

- REST, versioned via URL path (/api/v1/...); internal cross-service aggregation via GraphQL, not exposed externally at pilot.
- Response envelope: { data, meta, errors } — JSON:API-inspired, applied uniformly by the custom DRF exception_handler.
- Auth: Authorization: Bearer <JWT>; anonymous-allowed routes are explicitly whitelisted (Citizen Portal public reads, grievance submission).
- Standard error codes: 400 validation, 401 unauthenticated, 403 unauthorized (RBAC), 404 not found, 409 conflict, 422 business-rule violation, 429 rate-limited, 500 server error — every error body carries a machine-readable error_code for client-side i18n.
- GeoJSON is the wire format for every spatial response (FeatureCollection), never a custom coordinate array format.

7.2 Representative Endpoint Specifications

Auth Service

Endpoint	Method	Notes
/api/v1/auth/login	POST	{username,password} or OIDC redirect → {access_token, refresh_token, expires_in}
/api/v1/auth/refresh	POST	Refresh-token rotation; reuse detection invalidates the full token family
/api/v1/auth/me	GET	{user, role, permissions[]}

Asset Service

Endpoint	Method	Notes
/api/v1/assets	GET	Query: district_id, department_id, category_id, bbox → Facility[]
/api/v1/assets	POST / PATCH	RBAC: dept-write
/api/v1/assets/ingest	POST	Multipart CSV + department_id → 202 async {batch_id, status}
/api/v1/assets/ingest/{batch_id}	GET	{status, success_count, failure_count, errors[]}

Workflow & Grievance Service

Endpoint	Method	Notes
/api/v1/workflows	POST	{workflow_type, related_asset_id, ...} → 201 WorkflowInstance
/api/v1/workflows/{id}/transition	POST	{to_state, remarks}; role-gated per guard condition
/api/v1/grievances	POST	Anonymous allowed; {citizen_name, phone, asset_id, description, photo}
/api/v1/grievances/{id}/status	GET	Anonymous via tracking token

Enterprise Spatial Analytics & GIS Decision Support

Endpoint	Notes
/api/v1/analytics/accessibility	Isochrone / service-radius → GeoJSON FeatureCollection
/api/v1/analytics/suitability	solar-roof / solar-farm / water-recharge / school-site → features with suitability_score
/api/v1/analytics/watershed	Watersheds + drainage network → GeoJSON

Endpoint	Notes
/api/v1/decision-support/recommendations	district_id, decision_class → Recommendation[] with linked spatial evidence
/api/v1/gis/catalog	dataset_type, keywords, bbox → CatalogEntry[]
/api/v1/gis/remote-sensing/{ndvi lulc change-detection}	Raster tile URL / GeoJSON
/api/v1/gis/tiles/{z}/{x}/{y}.pbf	Vector tile endpoint, Redis/CDN-cached

7.3 Rate Limiting Tiers

Consumer Class	Limit
Anonymous (Citizen Portal)	60 req/min per IP
Registered Citizen	120 req/min
Line Department User	300 req/min
Admin/DM	600 req/min
Internal service-to-service	Not rate-limited (mTLS-authenticated)

7.4 Pagination, Filtering, Sorting, Swagger/OpenAPI

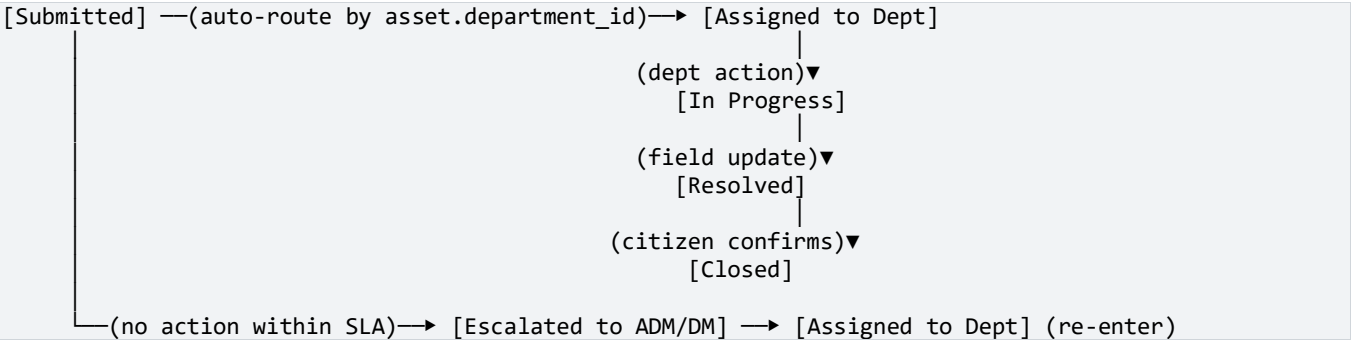
- List endpoints accept district_id/departement_id/category_id/bbox filters uniformly; page + page_size query params with meta.count in the response envelope.
- OpenAPI spec is generated per service via drf-spectacular and aggregated at the API Gateway; Swagger UI / Redoc exposed per-service in non-production environments only.
- The OpenAPI spec is the CI contract-diff source of truth (Ch. 26.3) — any breaking change without a version bump fails the build before merge.

8. Workflow Engine

8.1 Design Approach

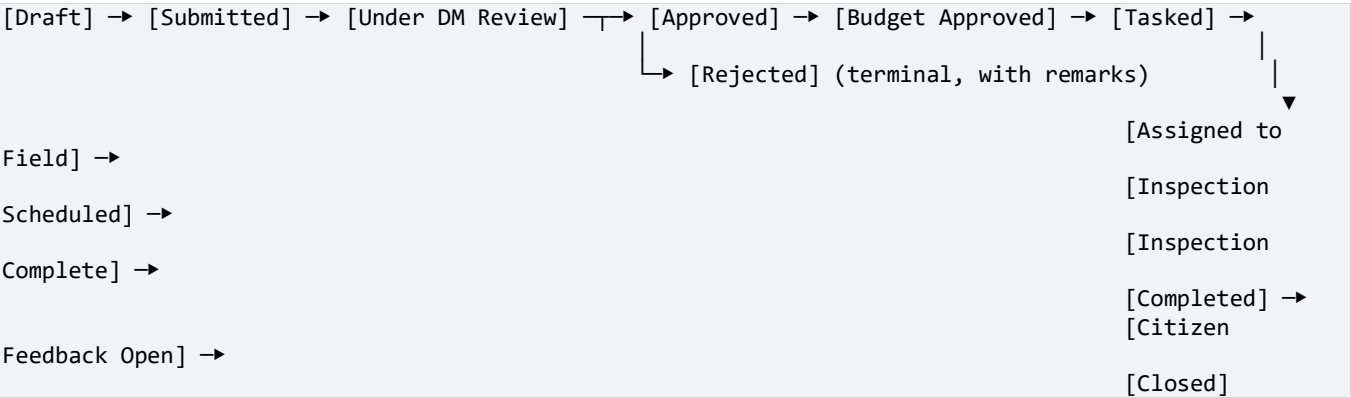
A single generic state-machine service (svc-workflow) parameterized by workflow_type, rather than one bespoke service per process. Each workflow type registers a state graph (states, allowed transitions, guard conditions, SLA timers) as configuration, enabling new department workflows — e.g. a future Solar subsidy approval — without a service redeploy.

8.2 Citizen Complaint Workflow



States: submitted → assigned → in_progress → resolved → closed, with escalated as a re-entrant side-state. SLA default: 14 days to resolved, configurable per asset category.

8.3 Department Proposal → DM Approval → Budget → Tasking



8.4 Guard Conditions (excerpt)

Transition	Guard Condition	Actor
draft → submitted	All mandatory proposal fields present	Department Officer
under_review → approved	actor.role ∈ {ADM, DM} AND actor.district_id = proposal.district_id	ADM/DM
approved → budget_approved	approved_amount ≤ actor.approval_limit (delegated limits table)	DM/State Admin
inspection_complete → completed	Geo-tagged photo evidence attached (geo_tag_variance_m < threshold)	Field Engineer

8.5 Escalation, Notification & Audit Hooks

- Every transition publishes workflow.transitioned to RabbitMQ — consumed independently by svc-notification (templated message), svc-audit (immutable log), and svc-analytics (recompute proposal-outcome features for the SDSS).
- SLA breach is evaluated by a scheduled job scanning txn_workflow_instance.sla_due_at; a breach transitions state to escalated and notifies the next role up the reports_to_user_id chain, capped at District Collector level for pilot.

8.6 Department-Specific Workflows (built on the same engine)

Department	Additional Workflow	States
Health	Disease Outbreak Alert	reported → verified → response_active → contained
Water	Maintenance Request	reported → dispatched → repaired → verified
Education	Enrolment Anomaly Alert	Routed to DEO for review on dropout-hotspot flag
Tourism	Circuit Planning	proposed → route_validated → published
Solar	Installation Commissioning	site_approved → installed → grid_connected → generating

9. Authentication & Authorization

9.1 Roles & Scope

Role	Scope
National Admin	All states/districts
State Admin	All districts within assigned state
District Collector/DM	Full control within assigned district, all departments
ADM	Delegated control within assigned district, assigned departments
Department Officer	Full control within assigned district + department
Field Engineer / Data Entry Operator	Asset CRUD + inspection updates within assigned district + department
Citizen (registered)	Own grievances only
Citizen (anonymous)	Public read-only

9.2 JWT & Session Design

- Access token: JWT via django-rest-framework-simplejwt, 15-minute TTL, carries sub, role, district_id, department_id, permissions[] (denormalized at issuance for fast gateway-side checks; forced re-auth on role change).
- Refresh token: 7 days, rotating, stored hashed in Redis; reuse detection invalidates the entire token family (breach mitigation).
- Concurrent-session cap configurable per role — Admin roles: 3 devices; Field Engineer mobile: 1 device, to reduce credential-sharing risk.

9.3 RBAC Enforcement — Defense in Depth

Evaluated at three independent layers, any one of which is sufficient to prevent a leak:

- API Gateway — coarse route-level RBAC check.
- Service layer — fine-grained row/field-level via PostgreSQL RLS + application-level HasRolePermission/IsDistrictScoped DRF permission-class checks.
- GIS proxy layer — CQL filter injection at svc-gis, scoping WFS/WMS access to the caller's district even on the shared GeoServer instance.

9.4 Government SSO

OIDC-compliant integration path reserved for a government IdP (NIC eOffice/DigiLocker-class), per LLD assumption A3; local username/password is the pilot-only fallback. The exact IdP endpoint is an open item requiring client confirmation before Phase 2 (Ch. 30.3).

9.5 RBAC Schema

- rbac_role — name, scope_level (national/state/district/department/self)
- rbac_permission — resource (asset, proposal, masterdata, ...), action (create/read/update/delete/approve)
- rbac_role_permission — role_id × permission_id
- rbac_user_district_scope — user_id × district_id × department_id (supports an ADM covering multiple districts — an explicit edge case in the LLD)

10. Search Architecture

10.1 Index Design (OpenSearch)

Primary index ndisp-facilities, synchronized from ast_facility via Debezium CDC over a RabbitMQ stream to an indexer worker.

```
{
  "id": "uuid",
  "name": "text (analyzed, multi-lingual)",
  "department": "keyword",
  "category": "keyword",
  "district": "keyword",
  "block": "keyword",
  "location": "geo_point",
  "status": "keyword",
  "attributes_flat": "object (dynamic)",
  "boost_score": "float"
}
```

10.2 Search Capabilities

- Global search — multi-field match across name/attributes_flat with department/category filters (powers Citizen Portal universal search).
- Spatial search — geo_distance and geo_bounding_box queries (“hospitals within 5km”, viewport-visible facilities).
- Autocomplete — edge n-gram analyzer on name, sub-100ms target.
- Geospatial catalog search — a secondary ndisp-gis-catalog index mirrors gis_catalog_entry so raster/vector/satellite datasets are discoverable through the same search experience, primarily for Admin Portal data-management views.

10.3 Relevance & Localization

boost_score weights active, high-confidence-geocoded, recently-verified facilities higher. Multi-lingual analyzers (English + Hindi ICU tokenizer). Index alias pattern ndisp-facilities-v{n} enables zero-downtime reindex-and-swap when adding new locale analyzers.

11. Document Management

11.1 Upload Pipeline

Client → svc-document (pre-signed MinIO URL, avoiding proxying large files through the gateway) → EXIF extraction (GPS, timestamp) → geo-tag variance check against the claimed asset location → virus scan (ClamAV sidecar) → metadata persisted to doc_file.

11.2 Geo-Tagged Images

Mandatory for: inspection completion evidence (a hard workflow guard condition), citizen grievance photos, and field-engineer asset condition updates. geo_tag_variance_m flags mismatches for QA review rather than silent rejection, since network/GPS drift is common in rural connectivity.

11.3 Versioning, OCR & Digital Signature

- doc_file.version increments on re-upload against the same owner_type + owner_id; prior versions retained, never overwritten — e.g. before/after repair photos.
- Tesseract OCR (async job) applied to scanned proposals and scheme-application documents, indexed into OpenSearch for document search.
- Digital signature reference (doc_file.signature_ref) reserved for government PKI (eSign/DSC) compatibility — integration deferred to Phase 2 pending a confirmed government PKI endpoint.

12. Reporting Engine

Format	Use Case	Generator
PDF	Executive summary, facility listing, approval sanction letter	WeasyPrint (HTML/CSS → PDF, pure Python)
Excel	Department asset export, KPI data dump	openpyxl
CSV	Raw data export for external analysis	Native streaming export
GIS Report	Map snapshot + legend + layer list	GeoServer WMS GetMap composited into PDF
Dashboard Report	Scheduled Admin KPI dashboard snapshot	Same PDF pipeline, templated per dashboard config
Executive Report	Cross-department district summary for DM/State Admin	Aggregates analytics + workflow + master-data via internal GraphQL

Generation is async job-queue driven for exports over 10k rows, with a completion notification and a pre-signed MinIO download link (24h expiry); synchronous generation is reserved for small on-demand reports with a sub-2-second render budget.

13. Notification System

Channel	Pilot	State/National Rollout
Portal (in-app)	Yes	Yes
SMS	Yes	Yes
Email	Yes	Yes
WhatsApp	Reserved	Yes
Push (mobile)	Reserved	Yes

- Event bus: every workflow transition, ingestion completion, and SLA breach publishes to `ndisp.events`; `svc-notification` binds a queue per channel-worker for independent scaling and isolated failure handling.
- Escalation follows the `reports_to_user_id` chain, capped at District Collector level for pilot.
- Templates (`ntf_template`) are locale-specific — a new language is new template rows, not a code change.

14. Security

14.1 OWASP Alignment

Target OWASP ASVS Level 2. Key controls: serializer-layer input validation (DRF Serializers), parameterized queries only (Django ORM QuerySets / GeoDjango spatial lookups — no raw string concatenation for spatial queries), output encoding to prevent XSS, and CSRF tokens for any cookie-based state-changing submission (mobile/SPA primarily use bearer tokens, reducing CSRF surface).

14.2 Encryption

- At rest: AES-256 for PostgreSQL (K8s storage-class-level transparent disk encryption), MinIO server-side encryption.
- In transit: TLS 1.2+ everywhere; internal service-mesh mTLS reserved for Phase 2 (Istio/Linkerd evaluation) — pilot uses NGINX-terminated TLS + K8s NetworkPolicy isolation as the interim control.

14.3 Secrets Management

Kubernetes Secrets backed by an external secrets operator syncing from a vault (HashiCorp Vault or cloud KMS — exact provider an open decision pending the client's NIC infrastructure). No secrets in source control or container images, under any circumstance.

14.4 GIS-Specific Security

GeoServer is never exposed directly — all WMS/WFS/WMTS traffic is proxied through svc-gis, which injects district-scope CQL filters based on the JWT district_id claim. This is the control that prevents a Line Department WFS-edit session from writing outside its own district on a shared GeoServer instance.

14.5 Input Validation — Spatial-Specific

All spatial input (GeoJSON from CSV geocoding or map-drawing tools) is validated for: valid SRID, ring closure / self-intersection (ST_IsValid), and coordinate bounds within India's geographic envelope — a sanity check against obviously erroneous geocodes, applied before anything reaches PostGIS.

14.6 Audit Trail & Rate Limiting

- aud_event_log is write-once, enforced by a database trigger rejecting UPDATE/DELETE on the table — application-level protection backstopped by DB-level enforcement.
- Rate limiting enforced at the API Gateway with Redis-backed sliding-window counters per the tiers in §7.3; abuse patterns (e.g. scraping the citizen search API) trigger temporary IP-level throttling.

15. DevOps

15.1 Containerization

Multi-stage Dockerfiles per service — a build stage with the full toolchain (GDAL headers, PROJ, compilers for GeoDjango/psycopg2), a runtime stage on python:3.12-slim (with GDAL/GEOS/PROJ system libraries layered in) for backend services, node:20-alpine for frontend build artifacts served via NGINX, and postgis/postgis for the database. Image size budget enforced in CI: under 400MB per Django service (GDAL/GEOS/PROJ system libraries push this above a typical slim-Python budget), under 200MB per frontend static image.

15.2 Kubernetes

- Namespace-per-environment: ndisp-dev, ndisp-staging, ndisp-prod.
- Helm charts parameterized per service; an umbrella chart composes the full platform.
- HPA on CPU + custom metrics — e.g. RabbitMQ queue depth drives scaling for svc-analytics and svc-notification, not just CPU.
- PodDisruptionBudgets preserve availability during node maintenance.

15.3 Reference Deployment Topology (per Ch. 8.4)

```
Kubernetes Cluster (Namespace: ndisp-prod)
├── Ingress (NGINX Ingress Controller + cert-manager)
├── Deployment: frontend-citizen (3 replicas, HPA 3-10)
├── Deployment: frontend-admin (2 replicas)
├── Deployment: frontend-linedept (2 replicas)
├── Deployment: api-gateway (3 replicas, HPA)
├── Deployment: svc-auth (3 replicas)
├── Deployment: svc-gis (4 replicas, HPA – CPU spatial ops)
├── Deployment: svc-workflow (3 replicas)
├── Deployment: svc-analytics (2 replicas, batch + on-demand)
├── Deployment: svc-notification (2 replicas)
├── Deployment: svc-master-data (2 replicas)
├── Deployment: svc-asset (3 replicas)
├── Deployment: svc-reporting (2 replicas)
├── Deployment: svc-search (2 replicas)
├── Deployment: svc-document (2 replicas)
├── Deployment: svc-audit (2 replicas)
├── StatefulSet: postgresql-postgis (primary + 2 read replicas, Patroni-managed)
├── StatefulSet: redis (Sentinel, 3 nodes)
├── StatefulSet: opensearch (3-node cluster)
├── StatefulSet: minio (4-node distributed erasure-coded)
├── StatefulSet: rabbitmq (3-node quorum queue cluster)
├── Deployment: geoserver (3 replicas behind internal LB)
└── Observability: prometheus, grafana, loki, otel-collector (DaemonSet)
```

15.4 Backup, DR & High Availability

- PostgreSQL: continuous WAL archiving to object storage + nightly base backup (pgBackRest); point-in-time recovery. Patroni-managed primary + 2 sync/async replicas with automatic failover.
- MinIO: erasure-coded 4-node cluster (survives 2-node loss); cross-region replication reserved for national rollout.
- Redis: Sentinel, 3-node. OpenSearch/RabbitMQ: 3-node clusters, quorum-based.
- RPO ≤ 15 min, RTO ≤ 4h — validated via quarterly DR drills (restore-to-staging exercise), not a one-time claim.

- Stateless services: minimum 2 replicas always, anti-affinity across nodes/AZs.

15.5 Horizontal Scaling Strategy

Stateless services scale via HPA. PostGIS scales read traffic via replicas — analytics/reporting reads route to the replica, transactional writes go to the primary. GeoServer scales horizontally behind an internal load balancer with a shared tile cache in Redis to avoid cache fragmentation across instances.

16. CI/CD

16.1 Pipeline (GitHub Actions)

```
on: push/PR
├── lint + typecheck (ruff/black/mypy for Django services, ESLint/tsc for frontend)
├── unit tests (per service, parallel matrix)
├── OpenAPI contract diff check (fails build on breaking change without version bump)
├── build + scan container image (Trivy)
├── integration tests (docker-compose: postgres, redis, rabbitmq, opensearch)
├── [main branch] push image to registry, tag = git SHA
└── [tag release] Helm upgrade → staging → manual approval gate → production
```

16.2 Branch Strategy & PR Rules

- Trunk-based development with short-lived feature branches; PRs require the full CI matrix green plus one approving review before merge.
- The contract-diff check is a hard merge blocker — no manual override path for breaking API changes without a version bump.
- Production deploys are tag-triggered with a manual approval gate after staging validation, never a direct push-to-prod path.

17. Testing Strategy

Type	Approach
Unit	Jest (frontend React components/hooks) + pytest / pytest-django (backend Django services); ≥ 80% coverage target on business-logic layers (services), not thin views/serializers
Integration	docker-compose environment spinning up real PostGIS/Redis/RabbitMQ/OpenSearch per CI run — spatial queries are tested against real PostGIS, never mocked
GIS	Geometry validity after ingestion, deficit-score correctness against hand-computed fixture datasets, WMS/WFS proxy scoping (a Line Department user cannot fetch another district's features via crafted CQL)
Performance	k6/Gatling load tests against NFR targets; spatial query p95 tested under simulated national-scale data volume (100M+ points) as a pre-Phase-3 scale gate
Security	OWASP ZAP automated scan in CI; annual third-party penetration test recommended before national rollout (pending client confirmation of the certification process)
UAT	Structured sign-off with the actual District Collector/DM office and named data custodians (Civil Surgeon Office, PHED, DEO, Tourism Office, BREDA) before Phase 0 → Phase 1 promotion

18. Monitoring & Observability

- Prometheus — service health, request rate/latency/error-rate per route, PostGIS query duration histograms, RabbitMQ queue depth.
- Grafana — per-service dashboards plus a composite “GIS Health” dashboard (tile-serve latency, GeoServer thread-pool utilization, spatial query p95).
- Loki — centralized structured JSON log aggregation, correlated via trace_id.
- OpenTelemetry — distributed tracing across API Gateway → services → PostGIS/OpenSearch, critical for diagnosing slow spatial queries spanning multiple services.
- GIS-specific monitoring — GeoServer WMS/WFS/WMTS latency and cache hit-ratio; PostGIS slow-query log (>1s) auto-captured with EXPLAIN ANALYZE attached for the on-call engineer.
- Audit log monitoring — anomaly alerting on aud_event_log (e.g. abnormal permission_denied spikes, mass export by a single user) feeding a lightweight rule set; full SIEM integration reserved for Phase 2.
- Health checks — every service exposes /healthz (liveness) and /readyz (readiness, verifying DB/cache/queue connectivity) per Kubernetes probe conventions.
- Alert thresholds are tied directly to NFR targets: API p95 > 300ms (non-spatial) or > 800ms (spatial) triggers a warning alert.

19. Performance Optimization

Layer	Cache	TTL
API Gateway response	Redis	30–120s (GET, non-personalized routes)
Vector tiles	Redis + CDN	5 min (dynamic layers), 24h (static admin boundaries)
Analytics gap-scores	Materialized view	Until next recompute (nightly + on-demand)
Master data	Redis + in-app memory	1h, invalidated on admin edit

- CDN fronts static frontend assets and cached WMTS base tiles, with an origin shield in front of GeoServer to absorb traffic spikes during high-visibility public events.
- Frontend: route-level code splitting; map layers load on-demand as toggled, not on initial page load.
- Database: PgBouncer (transaction mode) connection pooling in front of PostgreSQL to support 20+ microservice instances without exhausting max_connections; read replicas dedicated to reporting/analytics workloads.

20. Implementation Roadmap

Phase	Scope	Technical Readiness Gate
Phase 0 — Pilot POC	Nalanda/Rajgir, 5–6 depts, CSV-only ingestion	Core ingestion, GIS engine, deficit detection, citizen portal, DM approval workflow live
Phase 1 — Pilot Hardening	Same districts	Full RBAC, notifications, audit, reporting operational; UAT sign-off from data custodians
Phase 2 — State Rollout	All Bihar districts	Multilingual framework populated, UDISE+/NHP API adapters live, multi-district load-tested
Phase 3 — National Rollout	700+ districts	HA validated at 100M+ asset scale, AI recommendation model trained, cross-state schema partitioning validated, DR drills passed

20.1 Reserved Extension Points (built in now to avoid later redesign)

- attributes JSONB on assets — new department/category without a schema migration.
- field_schema on mst_asset_category — new CSV formats without a code change.
- Workflow-engine parameterization — new workflow types without new services.
- Notification channel abstraction — WhatsApp/Push added as new workers, not architecture changes.
- IoT/real-time telemetry hook reserved on ndisp.events for future live sensor integration (water flow, air quality) without redesigning the ingestion pipeline.
- AI Recommendation Service interface stable between rules-based (pilot) and ML-based (national) implementations.

20.2 Client Confirmations Required Before Phase 2

Per assumptions flagged throughout the LLD, the following require explicit client/government confirmation before state rollout: hosting/data-residency model, government SSO/IdP integration details, secrets-management platform, and the formal security certification process.

21. Recommended Team Structure

Role	Focus
Solution Architect	Overall coherence across Volumes 1–5; owns ADRs and cross-team design reviews
GIS/Spatial Architect	svc-gis, svc-analytics, GeoServer, spatial schema (Ch. 12.11–12.14), Enterprise Spatial Analytics Engine
Frontend Team	apps/ + packages/ui-components, map-core, api-client, i18n
Backend Team	services/ (Django/GeoDjango), per-domain module ownership split across department pods
GIS/Remote-Sensing Team	Raster processing chains (DEM/NDVI/LULC), GDAL/OGR/PROJ toolchain, watershed/terrain models
DevOps/Platform Team	Kubernetes, Helm, CI/CD, observability stack, DR drills
QA Team	Unit/integration/GIS/performance/security test suites; UAT coordination with data custodians
Project Manager	Phase gating, department-onboarding sequencing, client confirmations tracking (§20.2)

22. Coding Standards & Naming Conventions

22.1 Naming

- Database: mst_ / txn_ / ast_ / anl_ / doc_ / ntf_ / aud_ / rbac_ / gis_ prefixes are mandatory (§5.1) — enforced by a CI migration linter, not just code review.
- Services: svc-<domain>, one clear business-domain boundary per service (§4.7) — no service should own more than one prefix group's write path.
- API routes: /api/v1/<resource> plural nouns; sub-resources nested (e.g. /api/v1/assets/{id}/documents).
- React components: PascalCase; hooks: use<Thing>; Redux slices: <domain>Slice.

22.2 Backend Standards

- View → Service → Selector is not optional — no direct ORM/queryset access from a DRF View/ViewSet, ever.
- Every mutating endpoint publishes its domain event to ndisp.events via the AuditLogMiddleware + signal-receiver pattern, even if no other consumer exists yet.
- No raw SQL string concatenation for spatial queries — Django ORM / GeoDjango spatial lookups, or parameterized raw queries only.

22.3 GIS Standards

- SRID 4326 in, SRID 4326 stored — reprojection happens only at the tile-serving edge.
- Every new raster/vector dataset gets a gis_catalog_entry and a gis_data_provenance record before it is considered “published,” not just loaded.
- New spatial capabilities are added to the Enterprise Spatial Analytics Engine's shared catalog (§6.5), not reimplemented inside a department module.

22.4 API Standards

- GeoJSON for all spatial payloads; the {data, meta, errors} envelope for all non-spatial payloads.
- Breaking changes require a new /v{n} path plus a deprecation window on the old path — verified by the CI contract-diff check.

23. Closing Notes

This blueprint is a synthesis, not a new design — every folder structure, naming convention, and phasing decision above is traceable to a specific chapter in LLD Volumes 1–5, the GIS-Enriched HLD, the Minutes of Meeting (v2), or the Nalanda District Matrix. Where an item required an explicit assumption (deployment model, SSO provider, secrets platform, security certification process), that assumption is called out for client confirmation rather than silently decided — consistent with the LLD's own practice of flagging assumptions (A1–A7, A-GIS-1/2, A-SEC-1) rather than hiding them.

Confirmation: the backend stack in this blueprint is Python / Django / GeoDjango / Django REST Framework throughout, applied consistently across every backend folder structure, migration tool (Django migrations), and CI step in this document — GeoDjango's native PostGIS integration is what carries forward the GIS-first architectural principles (§1.1–1.3) into the implementation layer.